

CryAnalyze - Offline Deployment Blueprint

How CryAnalyze Achieves 100% Offline Edge Capability for Rural Healthcare

Team Foxfin: Abhijeet (Lead), Jatin Gupta, Siddhant Yadav | Track: Tech4Health

1. Why Offline Availability is Critical for Tech4Health

In rural primary healthcare centers (PHCs) and remote villages across India, internet connectivity is intermittent or non-existent. Traditional AI solutions that depend on cloud APIs (like OpenAI or AWS SageMaker) fail completely. CryAnalyze is engineered to run 100% offline directly on the user's laptop, smartphone, or local PHC tablet.

2. The 4 Technical Pillars of Offline Execution

Pillar 1: Browser Audio Capture (MediaRecorder API)

The React frontend captures microphone audio using the native HTML5 MediaRecorder API built directly into Chrome/Edge/Safari. It generates binary WAV blobs locally in the browser memory without sending audio data to external cloud storage.

Pillar 2: Local Feature Extraction (Librosa)

Instead of sending raw audio to cloud servers, the FastAPI backend uses Python's Librosa library locally. It resamples audio to 16 kHz and extracts 128-bin Mel-spectrogram matrices directly on the device CPU in under 0.3 seconds.

Pillar 3: Model Quantization (TensorFlow Lite / ONNX)

Heavy Keras models (.h5) are quantized into TensorFlow Lite (.tflite) or ONNX format using `export_offline_model.py`. Quantization compresses weights from 32-bit float to 8-bit integer, reducing model file size by ~4x (from 45MB to 8MB) and enabling lightning-fast CPU inference on low-cost devices.

Pillar 4: Localhost Microservice Architecture

FastAPI runs locally on loopback address 127.0.0.1:8000, while Vite/React serves static assets on 127.0.0.1:5173. All network calls occur over local IPC sockets. Zero internet packets are transmitted.

3. Offline Model Conversion Code (export_offline_model.py)

```
import tensorflow as tf
import os

def convert_to_tflite(model_path='models/cry_model.h5', output_path='models/cry_model.tflite'):
    model = tf.keras.models.load_model(model_path)
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    # Enable 8-bit integer quantization for offline CPU speed
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    tflite_model = converter.convert()
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, 'wb') as f:
        f.write(tflite_model)
    print('Quantized TFLite model generated successfully!')
```

4. How to Prove 100% Offline Capability to Hackathon Judges

Judges love live proof! Here is how Team Foxfin can prove offline capability during your presentation:

1. Disconnect WiFi completely: On stage, turn off your laptop's WiFi and Bluetooth.
2. Open localhost: Go to <http://localhost:5173> in Google Chrome.
3. Run Live Audio Record: Click the record button, speak/play a baby cry into the microphone, and click Analyze Cry.
4. Show Network Tab: Open Chrome Developer Tools (F12 -> Network tab) and show the judges that 100% of requests are routed to 127.0.0.1 with 0 bytes sent outside the machine.
5. Show Offline Model File: Open your project directory and point out models/cry_model.tflite running locally on Python CPU.

5. Standalone Desktop App Packaging (Optional Bonus)

For deployment to rural ASHA workers without Python installed, you can bundle the backend into a single executable using PyInstaller:

```
pip install pyinstaller
pyinstaller --onefile --add-data 'models;models' backend/main.py
```

This creates a standalone main.exe file that runs the entire FastAPI backend and ML model with a single double-click!